PAWGUARD BACKEND

Complete Review Preparation Guide

Built for the CEO / MD / Client Backend Review

What is built • Where it lives • Which APIs exist • How to explain it • Likely questions

FastAPI • Python 3.13 • PostgreSQL • SQLAlchemy (async) • Alembic • Redis + ARQ • JWT/Argon2

How to use this document

This PDF is a single study guide for the backend review. It maps every item on the first-week checklist to the exact module, file and API endpoint in the codebase, explains how each feature works internally, and gives you ready-to-say talking points and the most likely review questions. Read it top to bottom once today. On review day, keep the “Key files cheat-sheet” (section 14) and the “Checklist → Location map” (section 4) open. Everything here was read directly from the actual source code, so you can safely point at any file and explain it.

Folder conventions used in this guide:

- All paths are relative to the repository root `C:\Users\HP\pawguard-backend`.
- The versioned API lives under `/api/v1` (all routers are wired in `src/pawguard/api/v1/router.py`).
- Every module follows the architecture: `Router → Service → Repository → Database`.

1. Executive summary — one-paragraph pitch

PawGuard Backend is a **modular monolith** built with **Python 3.13 + FastAPI**, an **async SQLAlchemy** ORM over **PostgreSQL**, **Alembic** migrations, **Redis + ARQ** for caching, rate-limiting and background workers, **JWT (RS256)** authentication with Argon2id password hashing, MFA (TOTP), and a full Role-Based Access Control (RBAC) system. It follows **Domain-Driven Design and Clean Architecture**: every request flows Router → Service → Repository → Database. Business logic lives only in services. The same backend powers four clients — public website, admin portal, rescue staff mobile app and executive mobile app — and currently exposes **437+ endpoints across 27 modules**, with a 100% passed QA sweep and production performance tuned to a P95 of ~180–390 ms.

For this review the focus is the **first-week scope**: Pet Profiles & User Roles (Owner / Vet Clinic / Admin), the Mobile App shell backend, the QR Safety-Tag engine, the Lost Pet Alert System, the Vet Directory & Appointment Booking module, the Smart Reminders engine, and the QA / Performance & Security audit evidence.

2. Technology stack (memorize this table)

Layer	Technology	Why / notes for the review
Language	Python 3.13	Async-native, typed
API framework	FastAPI	OpenAPI auto-generated, async, Pydantic v2 validation
ORM	SQLAlchemy 2.x (async + asyncpg)	All I/O is async; repositories only do data access
Database	PostgreSQL 17 (Supabase in prod)	UUID PKs, JSONB, indexes, migrations
Migrations	Alembic	Versioned schema; drift-check in CI
Cache / Queue	Redis	Rate limiting, caching, ARQ job queue
Background jobs	ARQ worker (cron + on-demand)	Emails, push, lost-alert broadcast, reminders
Auth	JWT RS256 + Argon2id	Password hashing; asymmetric token signing
MFA	TOTP (pyotp)	Optional two-factor for accounts
Files	AWS S3 (presigned URLs)	Medical history uploads never touch our API body
Validation	Pydantic v2	Request/response models
Logging	structlog (structured)	Request ID, user ID, latency, status per request
Security scan	bandit	CI quality gate
CI	GitHub Actions	lint (ruff), mypy, bandit, pytest, alembic check, Docker build
Container	Docker / docker-compose	API + worker + Redis

3. System architecture

3.1 Request flow (the one diagram you must be able to draw)

```
Client (Flutter / Web / Admin)
        |
        v
   Router  (auth, authorise, validate, call service, respond)   <- src/pawguard/api/v1/*
        |
        v
   Service (owns ALL business logic)                            <- src/pawguard/modules/<mod>/service.py
        |
        v
   Repository (data access only, no decisions)                  <- src/pawguard/modules/<mod>/repository.py
        |
        v
   Database (PostgreSQL via SQLAlchemy async)
```

Rules enforced in AGENTS.md: business logic never lives in routers or repositories; SQL is never written in routes; every endpoint is production-ready with authentication, authorisation, input validation, business validation, execution and audit logging.

3.2 Folder map

```
pawguard-backend/
■■■ alembic/                    # DB migrations (versioned schema)
■■■ docs/                       # architecture PDFs, QA + performance reports
■■■ scripts/                    # seeders (roles/permissions, clinics, veterinarians)
■■■ secrets/                    # JWT key pair (dev)
■■■ src/pawguard/
■   ■■■ main.py                  # FastAPI app factory (mounts /api/v1, health, startup)
■   ■■■ api/v1/router.py         # aggregates ALL module routers
■   ■■■ core/                    # config, exceptions, pagination, rate limiter, responses, PII masks
■   ■■■ db/                      # session, base model, mixins (UUID pk, timestamps, soft delete), UoW
■   ■■■ modules/                 # 27 domain modules
■   ■   ■■■ auth/                # register/login/JWT/RBAC/MFA/OAuth/audit
■   ■   ■■■ companion_pet/       # pet profiles, QR tags, clinics, appointments, reminders
■   ■   ■■■ lost_found/          # lost/found reports, matching, claims, broadcast
■   ■   ■■■ notifications/       # in-app + push + email notifications
■   ■   ■■■ storage/             # S3 uploads (presigned URLs)
■   ■   ■■■ ... (20+ more: dog, medical, adoption, rescue, finance, ...)
■   ■■■ redis/                   # Redis client (fast failover when offline)
■   ■■■ services/                # cross-domain: audit, email, storage, notifications
■   ■■■ workers/                 # ARQ worker + cron + job modules
■■■ tests/                      # unit / integration / e2e / smoke / regression / load
■■■ pyproject.toml / uv.lock    # deps + quality-gate config
■■■ openapi.json                # full generated OpenAPI spec
```

4. First-week checklist → where it lives in the code

This is the mapping table for the review. If asked “where is the QR engine?” or “where is the lost pet broadcast?”, this is the answer.

Checklist item	Module	Primary files	Key endpoints
Pet Profiles & User Roles (Owner / Vet / Admin)	auth + companion_pet	modules/auth/*, modules/companion_pet/models.py	/api/v1/auth/*, /api/v1/companion-pets
Mobile App shell backend (auth, profile, medical uploads)	auth + companion_pet + storage	modules/auth/router.py, modules/companion_pet/router.py, modules/storage/*	/auth/register, /auth/login, /auth/me, /companion-pets/{id}/medical-files/upload-url
QR Safety Tag engine	companion_pet	modules/companion_pet/service.py (provision/scan), models.py (SafetyTag)	POST /companion-pets/{pet_id}/safety-tag, POST /companion-pets/safety-tag/scan
Repository commit & Day-20 QA report	git + tests + docs/qa	git log, tests/**, docs/qa/**	n/a (evidence files)
Lost Pet Alert System (broadcast, GPS pinning, QR scan flow)	lost_found + companion_pet + notifications	modules/lost_found/*, workers/jobs/lost_found_jobs.py	/lost-found/lost, /lost-found/lost/{id}/broadcast, /companion-pets/safety-tag/scan
Vet Directory & Appointment Booking	companion_pet	modules/companion_pet/*	/companion-pets/clinics, /clinics/{id}/veterinarians, /companion-pets/appointments
Smart Reminders (vaccination & medication push)	companion_pet + notifications + workers	modules/companion_pet/service.py, workers/jobs/companion_pet_jobs.py, arq_worker.py	/companion-pets/{pet_id}/reminders, cron 09:45 delivery
API Performance & Security Audit	docs/performance	docs/performance/final-report.md, docs/performance/baseline.md	442 endpoints, P95 180-390ms, 100% pass

5. Authentication & User Roles (Owner / Vet Clinic / Admin)

5.1 What is built

- **Registration, login, logout, refresh tokens** with JWT (RS256). Passwords hashed with Argon2id. Login is throttled (10/min) and audited.
- **Email verification** + **password reset** flows; verification/reset emails enqueued to the ARQ worker (never sent in the request path).
- **Session management** — list active sessions, revoke a session, log out all devices.
- **MFA** — TOTP enroll, confirm, verify, disable (with rate limits).
- **OAuth / social login** — Google/Apple ID-token login, link/unlink accounts (audience-verified, fails closed when unset).
- **RBAC** — 15 roles, ~90 permission codes, enforced by `require_permission(...)` dependency on every protected endpoint. Roles are seeded and reconciled at startup from `scripts/seed_roles_and_permissions.py`.
- **Audit trail** — every sensitive action records actor, IP, event type and metadata.

5.2 Roles relevant to this review

Role	Who	Permissions they carry (relevant)
super_admin	Admin (CEO/MD/client admin)	system:admin, all module permissions, companion_pet:* , safety_tag:manage, vet_clinic:manage, appointment:manage, lost_found:broadcast
app_user	Pet Owner (mobile app)	companion_pet:create/read/update/delete, medical_upload, safety_tag:manage, vet_clinic:read, appointment:create/read/cancel, lost_found:broadcast, public:*
general_public / donor	Public web / donors (owners too)	same owner-grade companion-pet permissions as app_user (seeded for the public portal)
veterinarian	Vet Clinic staff	medical:create/read/update, companion_pet:read/update, medical_upload, vet_clinic:read, appointment:read/manage
rescue_centre_admin / coordinators	Staff	operational permissions + companion_pet / vet clinic manage

Vet-clinic affiliation is a **data-level grant** (not a role): a user is added to a clinic through `vet_clinic_memberships` (role string ‘staff’/‘vet’). Clinic staff additionally get pet access through `pet_clinic_access`, which is auto-granted when an appointment is booked. This is a strong talking point: **access control is enforced in the service layer for every resource**, not just at the router.

5.3 Auth endpoints (auth module, prefix /api/v1/auth)

Method	Endpoint	Purpose
POST	/auth/register	Create account + queue verification email
POST	/auth/login	Login; returns JWT pair; MFA challenge if enabled
POST	/auth/mfa/verify	Complete MFA step of login
POST	/auth/refresh	Rotate access/refresh tokens
POST	/auth/logout	End current session
POST	/auth/logout-all	End all sessions
GET	/auth/me	Current user profile (app home screen)

Method	Endpoint	Purpose
PUT	/auth/me	Update profile (name, phone, photo, push token FCM, etc.)
GET	/auth/sessions	List active sessions
DELETE	/auth/sessions/{session_id}	Revoke one session
POST	/auth/password/change	Change password (logs out other sessions)
POST	/auth/password/reset/request	Send reset email
POST	/auth/password/reset/confirm	Apply reset token
POST	/auth/email/verify/confirm	Verify email with token
POST	/auth/mfa/enroll | /confirm | /disable	MFA management
POST	/auth/oauth/login	Google / Apple login
POST / GET / DELETE	/auth/oauth/*	OAuth accounts list / link / unlink

Key files: `src/pawguard/modules/auth/router.py` (endpoints), `service.py` (business logic, tokens, MFA, audit), `models.py` (User, Role, Permission, UserSession, audit log), `rbac.py` + `permission_codes.py` (enforcement + registry).

6. Pet Profiles & Medical History Uploads (companion_pet)

6.1 What is built

- Owners create **companion pet profiles**: name, species, breed, sex, birth date, color, microchip id, emergency notes, and a 'scan enabled' flag (controls whether their QR tag is scannable).
- Pets are **scoped to the owner** — an owner sees only their own pets; admins see all; clinic staff only see pets they have explicit clinic access to.
- **Medical history uploads** use **S3 presigned URLs**: the app asks the API for an upload URL, uploads the file directly to S3 (never through our server), then confirms. Files are stored as `stored_files` bound to the pet.
- **Medical records** can attach a stored file, record type, notes and an optional **next_reminder_at** that automatically creates a vaccination/medication reminder for the owner.

6.2 Pet-profile endpoints (companion_pet)

Method	Endpoint	Notes
POST	/companion-pets	Create pet (needs companion_pet:create)
GET	/companion-pets	List my pets (owner scoped; admin sees all); paginated
GET/PATCH/DELETE	/companion-pets/{pet_id}	Read / update / soft-delete a pet (owner or admin)
POST	/companion-pets/{pet_id}/medical-files/upload-url	Get presigned S3 upload URL
PUT	/companion-pets/{pet_id}/medical-files/{file_id}/confirm	Confirm upload is complete
GET	/companion-pets/{pet_id}/medical-files	List medical files (paginated)
POST	/companion-pets/{pet_id}/medical-records	Add medical record; auto-creates reminder if next_reminder_at set
GET	/companion-pets/{pet_id}/medical-records	List medical history
DELETE	/companion-pets/medical-records/{record_id}	Soft-delete a record

Key files: `modules/companion_pet/router.py` (endpoints), `service.py` (business rules, authorisation via _authorize_pet, audit events), `models.py` (CompanionPet, PetMedicalRecord), `schemas.py` (Pydantic), `repository.py` (SQL).

6.3 Database tables for pet profiles

Table	Purpose	Notable fields / rules
users	App identities + roles	email (unique), password_hash (argon2), is_active, fcm_token, push_notifications_enabled, extended profile fields
roles / permissions / role_permissions	RBAC	15 roles; permission codes like companion_pet:create, safety_tag:manage
user_sessions	Active sessions	user_id, device, is_active, expires_at; indexed (user_id, is_active, expires_at)
companion_pets	Pet profiles	owner_id FK users, microchip_id (unique), is_scan_enabled; index (owner_id, deleted_at)
pet_medical_records	Medical history	pet_id FK, clinic_id FK, stored_file_id FK, record_type, title, occurred_at
stored_files	Uploaded files (S3)	object_key, entity_type='companion_pet', entity_id=pet, mime_type, is_uploaded

Table	Purpose	Notable fields / rules
pet_safety_tags	QR tags	token_hash (unique), token_prefix, is_active, scan_count, last_scanned_at

7. Unique QR Code Generation engine (Safety Tags)

7.1 How it works (the most likely deep-dive question)

```
1. Owner taps 'Generate QR' in the app
   POST /api/v1/companion-pets/{pet_id}/safety-tag
   -> Service creates a random token: secrets.token_urlsafe(32)  (~43 chars)
   -> Stores ONLY the SHA-256 hash of the token + a visible prefix (first 8 chars)
   -> Returns the raw token ONE TIME (response field 'raw_token')
   -> The app encodes raw_token into the QR code printed on the pet's tag

2. Anyone scans the tag with their phone
   POST /api/v1/companion-pets/safety-tag/scan   (PUBLIC, rate-limited 20/min)
   -> body: { "token": "..." }
   -> server hashes the submitted token and looks it up (no raw tokens stored)
   -> returns pet name, species, breed, color, emergency notes + photo URL
   -> NO owner PII is returned (privacy-safe), and is_scan_enabled gates it
   -> increments scan_count and records last_scanned_at
```

7.2 Security design points (say these confidently)

- The raw token is **never persisted** — only a SHA-256 hash, so a DB leak cannot be used to forge scans.
- The token is **high-entropy** (256 bits) and **rotatable**: calling the provision endpoint again rotates the hash; only one active tag per pet (partial unique index).
- The scan endpoint is **public by design** (anyone with the tag can scan) but **rate-limited** and returns **zero owner PII**.
- Owners can switch off scanning via `is_scan_enabled` — scans then return ‘not found’.
- Every provision and every scan is written to the **audit trail** (SAFETY_TAG_PROVISIONED / SAFETY_TAG_SCANNED).

Method	Endpoint	Access	Notes
POST	/companion-pets/{pet_id}/safety-tag	Owner or admin (safety_tag:manage)	Provision / rotate; returns raw_token once
GET	/companion-pets/{pet_id}/safety-tag	Authorized reader	Metadata only — never returns the token
POST	/companion-pets/safety-tag/scan	Public + rate limit 20/60s	Hashed lookup; PII-free pet card + photo

Key files: `modules/companion_pet/service.py` (_new_tag_token, _hash_tag_token, provision_safety_tag, scan_safety_tag), `models.py` (SafetyTag), `schemas.py` (SafetyTagScanResponse). The shelter-facing dog module also renders an in-memory QR image (`GET /api/v1/dogs/{dog_id}/qr-image`), but the app-facing safety tag engine is the companion_pet one above.

8. Lost Pet Alert System

8.1 How it works

```
1. Owner reports a lost pet with GPS pin
   POST /api/v1/lost-found/lost
   body includes: pet_name, breed, color, location_address, latitude, longitude, lost_at
   -> report created as ACTIVE; auto-matching runs against found reports

2. Owner (or admin) triggers the community broadcast
   POST /api/v1/lost-found/lost/{report_id}/broadcast   (rate-limited 3/hour)
   -> Service validates ownership + ACTIVE status, then ENQUEUES an ARQ job
   -> Returns instantly (job runs in the background - TRANSACTION RULES)

3. ARQ worker job 'broadcast_lost_pet_alert' fans out
   -> loads the report with a row lock, skips if already broadcasted (idempotent)
   -> targets all active users except the reporter
   -> writes in-app notifications in batches of 500
   -> stamps broadcasted_at so retries never double-send

4. Community can view + report sightings / matches
   GET /lost-found/lost, /lost-found/found  (PII of reporters is masked for non-owners)
   GET /lost-found/lost/{id}/matches  -> matched found reports with reasons + distance
```

8.2 GPS location pinning

Both lost and found reports store `latitude` and `longitude` (Numeric(9,6)) plus a human-readable `location_address`. Matching computes **distance_km** and **temporal_gap_days** between a lost report and found reports, and persists **match_reasons** (JSONB) so the app can explain *why* a match was suggested (score transparency — PRR 3.10).

8.3 Ownership claim / verification flow

A matched party can **submit an ownership claim** (documents: microchip certificate, vet bill, photo proof), and staff with `system:admin` can **review/approve/reject**. Approval resolves both reports. Every step is audited.

Method	Endpoint	Notes
POST	/lost-found/lost	Report lost pet (GPS) - authenticated, rate-limited 10/min
GET	/lost-found/lost	Public list, reporter PII masked, filters by status/species
POST	/lost-found/lost/{report_id}/broadcast	Queue community broadcast (3/hour)
GET	/lost-found/lost/{report_id}/matches	Matched found reports (owner or public:read)
POST	/lost-found/found	Report found roaming animal
GET	/lost-found/found	Public list (masked)
POST	/lost-found/matches/{match_id}/claim	Submit ownership claim (rate-limited 5/hour)
POST	/lost-found/matches/{match_id}/claim/review	Admin verifies claim
GET	/lost-found/reunion-stories	Public success stories
DELETE / POST	/lost-found/lost|found/bulk/delete	Admin soft-delete

QR tag scan flow within lost pet UX: when someone finds a pet wearing a PawGuard tag, they scan it (section 7) and instantly see the pet card + emergency notes and “contact a local veterinary clinic” guidance — the finder can then also file a found report from the same screen. The scan endpoint returns `photo_url` and a message advising urgent care, without exposing the owner.

Key files: `modules/lost_found/router.py`, `service.py` (queue_lost_alert_broadcast, matching), `models.py` (LostReport, FoundReport, ReportMatch), `workers/jobs/lost_found_jobs.py` (broadcast job).

9. Interactive Vet Directory & Appointment Booking

9.1 What is built

- **Vet clinic directory** — clinics with name, address, phone, email, services, GPS coords, emergency flag. Listing is public (read-only) so the app booking screen works without forcing a login round-trip.
- **Veterinarian listing per clinic** — public endpoint returning the clinic's staff (vets) for the booking picker.
- **Appointment booking** — owner books against a clinic (+ optional vet) with start/end, reason, notes. Status lifecycle: requested → confirmed → completed / cancelled / no_show.
- **Conflict prevention** — the service checks for overlapping appointments at the clinic before insert and re-checks on IntegrityError (double-insert safe).
- **Auto clinic access** — booking a pet into a clinic automatically grants the clinic access to that pet's record (so the clinic can read medical history), unless it was already granted.
- Clinic management (create/update/delete, membership) is **admin-only** (vet_clinic:manage).

Method	Endpoint	Access	Notes
GET	/companion-pets/clinics	Public	Directory listing, search + pagination
GET	/companion-pets/clinics/{clinic_id}/veterinarians	Public	Vets available for booking
POST	/companion-pets/appointments	Owner (appointment:create)	Book; conflict-checked
GET	/companion-pets/appointments	Owner / clinic staff / admin	Filter by clinic_id or pet_id
GET	/companion-pets/appointments/{appointment_id}	Owner / clinic member	Detail
POST	/companion-pets/appointments/{id}/cancel	Owner / clinic / admin	With reason
POST	/companion-pets/appointments/{id}/confirm	Clinic staff (appointment:manage)	Confirm
POST	/companion-pets/clinics	Admin only	Create clinic
PATCH/DELETE	/companion-pets/clinics/{clinic_id}	Admin only	Update / soft-delete
POST	/companion-pets/clinics/{clinic_id}/memberships	Admin only	Add clinic staff

Key files: `modules/companion_pet/router.py` (endpoints), `service.py` (create_appointment conflict logic, add_membership), `models.py` (VetClinic, ClinicMembership, PetClinicAccess, PetAppointment with CheckConstraint ends_at > starts_at). Seeder: `scripts/seed_veterinary_partners.py` and `seed_veterinarians.py`.

10. Automated Smart Reminders Engine

10.1 Two reminder paths

A. Owner-facing vaccination / medication reminders (companion_pet)

```
Manual:  POST /api/v1/companion-pets/{pet_id}/reminders   (kind=vaccination|medication, due_at, source_key)
Auto:    POST /companion-pets/{pet_id}/medical-records with 'next_reminder_at' (+ reminder_kind)
         -> service auto-creates a PetReminder, source_key = 'medical_record:<id>:<kind>' (unique)

Delivery (daily cron):  ARQ cron 09:45 -> send_companion_pet_reminders
   -> picks reminders whose due_at is within the next 24h (delivers ~1 day early)
   -> deliver_reminder_once():
       1. writes a ReminderDelivery row (unique reminder_id+user_id+scheduled_for)
       2. creates an in-app Notification (notification_type=companion_pet_vaccination|medication)
       3. sends an FCM push via NotificationService when the owner has push enabled + valid FCM token
   -> the unique row makes re-runs / retries idempotent (no duplicate pushes)
```

B. Staff-facing vaccination renewal reminders (medical module)

A separate cron (**09:30 daily**) runs `check_vaccination_renewals`, which notifies staff when any vaccination's `next_due_at` is within 14 days. Other crons cover inventory low-stock/expiry and post-adoption follow-ups.

Method	Endpoint	Notes
POST	/companion-pets/{pet_id}/reminders	Create reminder (duplicate source_key rejected)
GET	/companion-pets/{pet_id}/reminders	List pet reminders
DELETE	/companion-pets/{pet_id}/reminders/{reminder_id}	Soft-delete reminder
cron 09:45	send_companion_pet_reminders (ARQ)	Deliver due-in-24h reminders once, in-app + push

Key files: `modules/companion_pet/service.py` (create_reminder, deliver_reminder_once), `models.py` (PetReminder, ReminderDelivery with unique delivery key), `workers/jobs/companion_pet_jobs.py`, `workers/arq_worker.py` (cron registration), `modules/notifications/service.py` (in-app + FCM push).

11. Background workers & schedules (say ‘nothing blocking the HTTP request’)

Job	Schedule / trigger	What it does
send_companion_pet_reminders	cron 09:45 daily	Vaccination & medication reminders to owners (in-app + push)
check_vaccination_renewals	cron 09:30 daily	Staff alerts for vaccinations due in 14 days
check_inventory_low_stock	cron 00:00 & 12:00	Low-stock staff alerts
check_inventory_expiry	cron 09:00 daily	Expiry warnings (60 days)
post_adoption_followups	cron 10:00 daily	30/90/180-day post-adoption prompts
process_sponsorship_charges	cron 08:00 daily	Monthly sponsorship billing
broadcast_lost_pet_alert	on-demand (enqueued)	Fan-out lost alert to active users (batches of 500, idempotent)
send_*_email_job	on-demand (enqueued)	Verification, reset, notification emails

Worker entry point: `uv run arq pawguard.workers.arq_worker.WorkerSettings`. If Redis is unreachable the app still serves HTTP and degrades to a no-op worker (no crashes).

12. Repository commit & Day-20 QA execution evidence

12.1 Test suite (what proves it works)

Suite	Location	Covers
Unit	tests/unit/	Services, RBAC, MFA, reminders, lost-found matching, jobs (test_companion_pet.py, test_lost_found.py, test_scheduled_jobs.py, ...)
Integration	tests/integration/	API flows incl. companion_pet API, auth flows, storage, public access
E2E / API	tests/e2e/	Full endpoint execution incl. test_companion_pets.py, test_lost_found.py; results in tests/e2e/test_results.txt
Smoke	tests/smoke/	Health + happy paths
Regression	tests/regression/	PRR acceptance checks
Load	tests/load/locustfile.py	Locust performance scenarios

Run everything with `uv run pytest --cov=src/`. Quality gates: `uv run ruff check`, `uv run mypy src/`, `uv run bandit -c pyproject.toml -r src/`. CI enforces all of these on every PR.

12.2 QA artifacts already produced

Artifact	Location	Headline numbers
Endpoint inventory / QA run	docs/qa/all-endpoints.md / .json / .csv	437 endpoints across 27 modules
API test report (local)	docs/api_test_report_http_127.0.0.1_8000.json	Executed local run
API test report (Render)	docs/api_test_report_https_...render.com.json	Executed against live staging
Perf baseline + final report	docs/performance/final-report.md	442 endpoints, 100% pass, P95 180-390ms
OpenAPI spec	openapi.json	Full machine-readable API spec (~1.5 MB)
Postman collection	PawGuard.postman_collection.json	Ready-made manual test requests

12.3 Recent commits to cite

```
8f86e95 fix(companion-pets): make clinic veterinarians endpoint public (Flutter booking screen)
f3da5b1 chore: clean repo, add production seeders for vet directory, veterinarians, mobile appointments
99a8430 feat(companion-pets): veterinarian listing endpoint for appointment booking
36ef7ff fix(rbac): grant companion_pet read/create to app_user role
dd06c0d perf: optimize performance across 442 endpoints (auth caching, pool, pagination)
98f383a feat: smart reminders, QR scan photos, FCM push, auto-reminder from medical records
```

13. API Performance & Security audit summary

13.1 Performance (docs/performance/final-report.md)

Metric	Value
Registered / audited endpoints	442 / 442 (100%)
Actually executed	441 / 442 (1 blocked: live MFA/OAuth provider, documented)
Passed acceptance	441 / 442 (100% of executable)
Latency < 500 ms	441 / 441 (100%)
P95 after optimisation	~180 ms - 390 ms (was 850 ms - 2,800 ms)
Latency reduction	~75% - 85%

13.2 What was optimised (be able to name all four)

- **Auth/RBAC query elimination** — cached the active session onto CurrentUser; removed duplicate role/permission lookups (saved 40-80 ms on every authenticated call).
- **Pagination sort-order fix** — COUNT query now runs before ORDER BY in all 12 repository modules (1,200-2,500 ms → 140-340 ms on list endpoints).
- **Connection pool & timeouts** — pool_recycle=1800, pool_timeout=30 to survive PgBouncer idle drops.
- **Composite indexes + Redis failover** — migration e1f2a3b4c5d6 added high-traffic composite indexes; Redis client fast-fails (0.1 s timeouts) to a no-op when offline.

13.3 Security posture (top talking points)

- Passwords: **Argon2id**; tokens: **JWT RS256** with refresh-token rotation and reuse detection.
- Every protected endpoint: **authentication** → **authorisation (RBAC permission)** → **input validation (Pydantic)** → **business validation** → **execution** → **audit**.
- Resource-level access control in services (owner / clinic-membership / admin) — e.g. _authorize_pet.
- Rate limiting on auth, scan, broadcast, report and claim endpoints (Redis-backed, graceful offline).
- PII masking on public lost/found listings; QR scan returns zero owner PII.
- Raw QR tokens never stored (SHA-256 only); presigned S3 uploads; no secrets in repo; bandit in CI.
- Error contract: no SQL errors / stack traces leak to clients — standard ApiResponse error model.

14. Key files cheat-sheet (keep this page open during the review)

File	Why it matters
src/pawguard/main.py	App factory; mounts /api/v1, health endpoints, seed reconciliation on startup
src/pawguard/api/v1/router.py	Single place all 27 module routers are registered
modules/auth/router.py	All auth endpoints (register/login/me/MFA/OAuth/password)
modules/auth/service.py	Auth business logic: tokens, argon2, MFA, sessions, audit
modules/auth/rbac.py + permission_codes.py	RBAC enforcement dependency + permission registry
modules/companion_pet/router.py	Pet profiles, medical, QR, clinics, appointments, reminders endpoints
modules/companion_pet/service.py	Pet business rules, QR provision/scan, appointment conflicts, reminders
modules/companion_pet/models.py	companion_pets, vet_clinics, memberships, access, safety tags, reminders, appointments
modules/lost_found/router.py + service.py	Lost/found reports, matching, claims, broadcast queue
workers/jobs/lost_found_jobs.py	Idempotent fan-out broadcast job (batches of 500)
workers/jobs/companion_pet_jobs.py	Daily reminder delivery job
workers/arq_worker.py	All jobs + cron schedules
modules/notifications/service.py	In-app notifications + FCM push + email enqueue
modules/storage/*	S3 presigned uploads for medical files
scripts/seed_roles_and_permissions.py	The 15 roles and their permissions (RBAC source of truth)
scripts/seed_veterinary_partners.py	Vet directory seed data
docs/qa/*	Day-20 QA evidence (437 endpoints)
docs/performance/final-report.md	Performance & security audit (442 endpoints, P95 180-390ms)
openapi.json / PawGuard.postman_collection.json	Live spec + manual test collection

Startup seed reconciliation: roles and permissions are re-synced on every app start, so new permission codes ship to an already-seeded database without manual SQL.

15. Likely review questions + ready answers

Question	Answer to say
How is the backend structured?	Modular monolith, Clean Architecture: Router → Service → Repository → DB. Business logic only in services (AGENTS.md engineering constitution). 27 modules, 437+ endpoints.
How does authentication work?	Email/password + JWT RS256 access & refresh tokens, Argon2id hashing, optional TOTP MFA, OAuth (Google/Apple), session management, all behind rate limits and audit logging.
How are Owner / Vet Clinic / Admin roles enforced?	RBAC permission codes per endpoint (require_permission) PLUS data-level checks in services (pet owner, clinic membership, pet_clinic_access, admin override). Roles seeded from seed_roles_and_permissions.py.
Where is the QR engine and how does it work?	modules/companion_pet. Random 256-bit token, only SHA-256 stored, prefix for support, one active tag per pet, rotatable, public rate-limited scan returns PII-free pet card + photo.
How do medical history uploads work?	App requests a presigned S3 URL, uploads directly to S3, then confirms. Files are linked to the pet in stored_files; records can auto-create reminders.
How does the lost pet broadcast work?	POST /lost-found/lost/{id}/broadcast enqueues an ARQ job; worker fans out in-app notifications to active users in batches of 500; broadcasted_at + row lock make it idempotent.
How is GPS used in lost pet alerts?	lat/lng stored on reports; matching computes distance_km + temporal_gap_days and persists match_reasons for transparency.
How does appointment booking prevent double-booking?	Service checks for overlapping clinic appointments before insert and re-raises ConflictError on unique/IntegrityError race; CheckConstraint ends_at > starts_at.
How do smart reminders get delivered?	Daily cron 09:45 picks reminders due within 24h; deliver_reminder_once writes a unique delivery row then in-app notification + FCM push; unique key makes retries idempotent.
How do you know the API is fast and secure?	Performance audit: 442 endpoints, 100% pass, P95 180-390ms after 4 optimisations (auth caching, count-before-sort pagination, pool tuning, composite indexes). Security: Argon2, RS256 JWT, RBAC, rate limits, PII masking, hashed QR tokens, no secrets in repo, bandit + ruff + mypy in CI.
What would you improve next?	More push/email infrastructure hardening, per-tenant feature flags, OpenAPI example payloads, cache layer for the vet directory, and expanded load-test thresholds.

Final tip: if you don't know a detail, never guess — say "let me point you to the exact file" and open it. Every claim in this guide is directly backed by the source code paths listed in section 14.